$L' = M(P')$ for some DPDA P'.[5] By Theorem 6.20, there is an unambiguous grammar G' generating the language $N(P')$, which is L'.

Now, construct from G' a grammar G such that $L(G) = L$. To do so, we have only to get rid of the endmarker \$ from strings. Thus, treat \$ as a variable of G, and introduce production $\$ \to \epsilon$; otherwise, the productions of G' and G are the same. Since $L(G') = L'$, it follows that $L(G) = L$.

We claim that G is unambiguous. In proof, the leftmost derivations in G are exactly the same as the leftmost derivations in G', except that the derivations in G have a final step in which \$ is replaced by ϵ. Thus, if a terminal w string had two leftmost derivations in G, then $w\$$ would have two leftmost derivations in G'. Since we know G' is unambiguous, so is G. □

6.4.5 Exercises for Section 6.4

Exercise 6.4.1: For each of the following PDA's tell whether or not it is deterministic. Either show that it meets the definition of a DPDA or find a rule (or rules) that violates it.

a) The PDA of Example 6.2.

* b) The PDA of Exercise 6.1.1.

c) The PDA of Exercise 6.3.3.

Exercise 6.4.2: Give deterministic pushdown automata to accept the following languages:

a) $\{0^n1^{m+n}1^m | m, n \geq 0\}$

b) $\{w \in \{0, 1\}^* |$ the number of 0's is less than twice the number of 1's$\}$

c) $\{0^n1^m0^n \mid n$ and m are arbitrary$\}$.

Exercise 6.4.3: We can prove Theorem 6.19 in three parts:

* a) Show that if $L = N(P)$ for some DPDA P, then L has the prefix property.

! b) Show that if $L = N(P)$ for some DPDA P, then there exists a DPDA P' such that $L = L(P')$.

*! c) Show that if L has the prefix property and is $L(P')$ for some DPDA P', then there exists a DPDA P such that $L = N(P)$.

[5] The proof of Theorem 6.19 appears in Exercise 6.4.3, but we can easily see how to construct P' from P. Add a new state q that P' enters whenever P is in an accepting state and the next input is \$. In state q, P' pops all symbols off its stack. Also, P' needs its own bottom-of-stack marker to avoid accidentally emptying its stack as it simulates P.

!! **Exercise 6.4.4:** Show that the language

$$L = \{0^n1^n \mid n \geq 1\} \cup \{0^n1^{2n} \mid n \geq 1\}$$

is a context-free language that is not accepted by any DPDA. *Hint*: Show that there must be two strings of the form 0^n1^n for different values of n, say n_1 and n_2 that cause a hypothetical DPDA for L to enter the same ID after reading both strings. Intuitively, the DPDA must erase from its stack almost everything it placed there on reading the 0's, in order to check that it has seen the same number of 1's. Thus, the DPDA cannot tell whether or not to accept next after seeing n_1 1's or after seeing n_2 1's.

6.5 Summary of Chapter 6

- *Pushdown Automata*: A PDA is a nondeterministic finite automaton coupled with a stack that can be used to store a string of arbitrary length. The stack can be read and modified only at its top.
- *Moves of a Pushdown Automata*: A PDA chooses its next move based on its current state, the next input symbol, and the symbol at the top of its stack. It may also choose to make a move independent of the input symbol and without consuming that symbol from the input. Being nondeterministic, the PDA may have some finite number of choices of move; each is a new state and a string of stack symbols with which to replace the symbol currently on top of the stack.
- *Acceptance by Pushdown Automata*: There are two ways in which we may allow the PDA to signal acceptance. One is by entering an accepting state; the other by emptying its stack. These methods are equivalent, in the sense that any language accepted by one method is accepted (by some other PDA) by the other method.
- *Instantaneous Descriptions*: We use an ID consisting of the state, remaining input, and stack contents to describe the "current condition" of a PDA. A transition function $\vdash$ between ID's represents single moves of a PDA.
- *Pushdown Automata and Grammars*: The languages accepted by PDA's either by final state or by empty stack, are exactly the context-free languages.
- *Deterministic Pushdown Automata*: A PDA is deterministic if it never has a choice of move for a given state, input symbol (including ϵ), and stack symbol. Also, it never has a choice between making a move using a true input and a move using ϵ input.
- *Acceptance by Deterministic Pushdown Automata*: The two modes of acceptance — final state and empty stack — are not the same for DPDA's. Rather, the languages accepted by empty stack are exactly those of the languages accepted by final state that have the prefix property: no string in the language is a prefix of another word in the language.

(b) If $\delta(q, b, X)$ contains (p, γ), where b is in T or $b = \epsilon$, then

$$\delta'((q, bx), \epsilon, X)$$

contains $((p, x), \gamma)$. That is, P' always has the option of simulating a move of P, using the front of its buffer. If b is a symbol in T, then the buffer must not be empty, but if $b = \epsilon$, then the buffer can be empty.

3. Note that, as defined in (7.1), the start state of P' is (q_0, ϵ); i.e., P' starts in the start state of P with an empty buffer.

4. Likewise, the accepting states of P', as per (7.1), are those states (q, ϵ) such that q is an accepting state of P.

The following statement characterizes the relationship between P' and P:

- $(q_0, h(w), Z_0) \overset{*}{\underset{P}{\vdash}} (p, \epsilon, \gamma)$ if and only if $((q_0, \epsilon), w, Z_0) \overset{*}{\underset{P'}{\vdash}} ((p, \epsilon), \epsilon, \gamma)$.

The proofs in both directions are inductions on the number of moves made by the two automata. In the "if" portion, one needs to observe that once the buffer of P' is nonempty, it cannot read another input symbol and must simulate P, until the buffer has become empty (although when the buffer is empty, it may still simulate P). We leave further details as an exercise.

Once we accept this relationship between P' and P, we note that P accepts $h(w)$ if and only if P' accepts w, because of the way the accepting states of P' are defined. Thus, $L(P') = h^{-1}(L(P))$. $\square$

7.3.6 Exercises for Section 7.3

Exercise 7.3.1 : Show that the CFL's are closed under the following operations:

* a) *init*, defined in Exercise 4.2.6(c). *Hint*: Start with a CNF grammar for the language L.

*! b) The operation L/a, defined in Exercise 4.2.2. *Hint*: Again, start with a CNF grammar for L.

!! c) *cycle*, defined in Exercise 4.2.11. *Hint*: Try a PDA-based construction.

Exercise 7.3.2: Consider the following two languages:

$$L_1 = \{a^n b^n c^m | m, n \geq 0\}$$
$$L_2 = \{a^n b^m c^n | m, n \geq 0\}$$

a) Show that each of these languages is context free by giving grammars for each.

! b) Is $L_1 \cap L_2$ a CFL? Justify your answer.

!! **Exercise 7.3.3:** Show that the CFL's are *not* closed under the following operations:

* a) *min*, as defined in Exercise 4.2.6(a).

b) *max*, as defined in Exercise 4.2.6(b).

c) *half*, as defined in Exercise 4.2.8.

d) *alt*, as defined in Exercise 4.2.7.

Exercise 7.3.4: The *shuffle* of two strings w and x is the set of all strings that one can get by interleaving the positions of w and x in any way. More precisely, *shuffle*(w, x) is the set of strings z such that

1. Each position of z can be assigned to w or x, but not both.
2. The positions of z assigned to w form w when read from left to right.
3. The positions of z assigned to x form x when read from left to right.

For example, if $w = 01$ and $x = 110$, then *shuffle*$(01, 110)$ is the set of strings $\{01110, 01101, 10110, 10101, 11010, 11001\}$. To illustrate the necessary reasoning, the third string, 10110, is justified by assigning the second and fifth positions to 01 and positions one, three, and four to 110. The first string, 01110 has three justifications. Assign the first position and either the second, third, or fourth to 01, and the other three to 110. We can also define the shuffle of languages, *shuffle*(L_1, L_2) to the the union over all pairs of strings, w from L_1 and x from L_2, of *shuffle*(w, x).

a) What is *shuffle*$(00, 111)$?

* b) What is *shuffle*(L_1, L_2) if $L_1 = L(\mathbf{0}^*)$ and $L_2 = \{0^n 1^n \mid n \geq 0\}$.

*! c) Show that if L_1 and L_2 are both regular languages, then so is

$$shuffle(L_1, L_2)$$

Hint: Start with DFA's for L_1 and L_2.

! d) Show that if L is a CFL and R is a regular language, then *shuffle*(L, R) is a CFL. *Hint*: start with a PDA for L and a DFA for R.

!! e) Give a counterexample to show that if L_1 and L_2 are both CFL's, then *shuffle*(L_1, L_2) need not be a CFL.

*!! **Exercise 7.3.5:** A string y is said to be a *permutation* of the string x if the symbols of y can be reordered to make x. For instance, the permutations of string $x = 011$ are 110, 101, and 011. If L is a language, then *perm*(L) is the set of strings that are permutations of strings in L. For example, if $L = \{0^n 1^n \mid n \geq 0\}$, then *perm*$(L)$ is the set of strings with equal numbers of 0's and 1's.

a) Give an example of a regular language L over alphabet $\{0, 1\}$ such that *perm*(L) is not regular. Justify your answer. *Hint*: Try to find a regular language whose permutations are all strings with an equal number of 0's and 1's.

b) Give an example of a regular language L over alphabet $\{0, 1, 2\}$ such that *perm*(L) is not context-free.

c) Prove that for every regular language L over a two-symbol alphabet, *perm*(L) is context-free.

Exercise 7.3.6: Give the formal proof of Theorem 7.25: that the CFL's are closed under reversal.

Exercise 7.3.7: Complete the proof of Theorem 7.27 by showing that

$$(q_P, w, Z_0) \overset{*}{\underset{P}{\vdash}} (q, \epsilon, \gamma)$$

if and only if $((q_P, q_A), w, Z_0) \overset{*}{\underset{P'}{\vdash}} ((q, p), \epsilon, \gamma)$ and $p = \hat{\delta}(p_A, w)$.

7.4 Decision Properties of CFL's

Now, let us consider what kinds of questions we can answer about context-free languages. In analogy with Section 4.3 about decision properties of the regular languages, our starting point for a question is always some representation of a CFL — either a grammar or a PDA. Since we know from Section 6.3 that we can convert between grammars and PDA's, we may assume we are given either representation of a CFL, whichever is more convenient.

We shall discover that very little can be decided about a CFL; the major tests we are able to make are whether the language is empty and whether a given string is in the language. We thus close the section with a brief discussion of the kinds of problems that we shall later show (in Chapter 9) are "undecidable," i.e., they have no algorithm. We begin this section with some observations about the complexity of converting between the grammar and PDA notations for a language. These calculations enter into any question of how efficiently we can decide a property of CFL's with a given representation.